

Preventing Buffer Overflow Attacks Using GDB

This article is aimed at creating awareness about the need for secure coding practices to prevent buffer overflow attacks. It also demonstrates the use of the GNU Debugger (GDB) to detect buffer overflows in a program.



A buffer overflow is a critical flaw wherein a program accesses memory not allocated to a buffer, overrunning its size. If a buffer is given more input than its size, the memory location(s) immediately following the buffer will be overwritten. This leads to loss of data in these locations, and also potentially enables a malicious user to get higher privileges in the system. Buffer overflows are commonly found in programs written in C and C++, since these languages do not have built-in mechanisms for protection against buffer overflows. Buffer overflows are common in cases that involve copying strings from one buffer to another. Some commonly used and vulnerable functions include *gets*, *sprintf*, *strcpy* and *strcat*. These functions do

not perform bounds checking (i.e., whether the length of the string to be copied is smaller than the destination string) when copying the characters into another string.

Why it is important to understand buffer overflows

Buffer overflows have often been exploited to gain unauthorised access. One of the most sensational cases of hacking, involving buffer overflows, is the Morris worm incident that took place 25 years ago. Ever since, there have been similar cases of buffer overflows being exploited. Thus, there is a pressing need to prevent the possibility of buffer overflows because, if exploited, the